LECTURE NOTES

Lecture 9 - RNNs

AI506 — Advanced Machine Learning

Simon Holm

March, 2026

SDU

DEPARTMENT OF MATHEMATICS
AND COMPUTER SCIENCE

Contents

Text 3
  Vocabulary construction 3
  How to obtain Embeddings? 3
Recurrent Neural Networks 3
  RNNs share same weights across Time Step 4
  Problem of Long-Term Dependencies 4
Backpropagation-though-time 4
Three design patterns of RNNs 5
  Design 1 5
    Forward Calculation 5
    Gradient Calculation (backward) 5
  Design 2 7
    Recurrence from Output to Hidden 8
  Design 3 8
Teacher Forcing 8
LSTM (long short-term memory) 9

Text

Can be Can be interpreted as a sequence of words (or **tokens**)

Classic NLP tasks: Text categorization, sentiment analysis, translation

Vocabulary construction

build words using symbols/letters, then indicise words as **tokens**

$$\left[\underbrace{\text{The}}_{0}, \underbrace{\text{quick}}_{1}, \underbrace{\text{blue}}_{2}, \underbrace{\text{fox}}_{3}, \underbrace{\text{is}}_{4}\right]$$

Now 'The fox is quick' $= [0, 3, 4, 1]$

How to obtain Embeddings?

- **Old way**: load pre-computed word embeddings (e.g. word2vec, fastText) [King-Queen]
- **Modern way**: Learn word embeddings jointly with the main task.
  - Start with random word embeddings and then learn word embeddings in the same way you learn the other weights of the neural network.

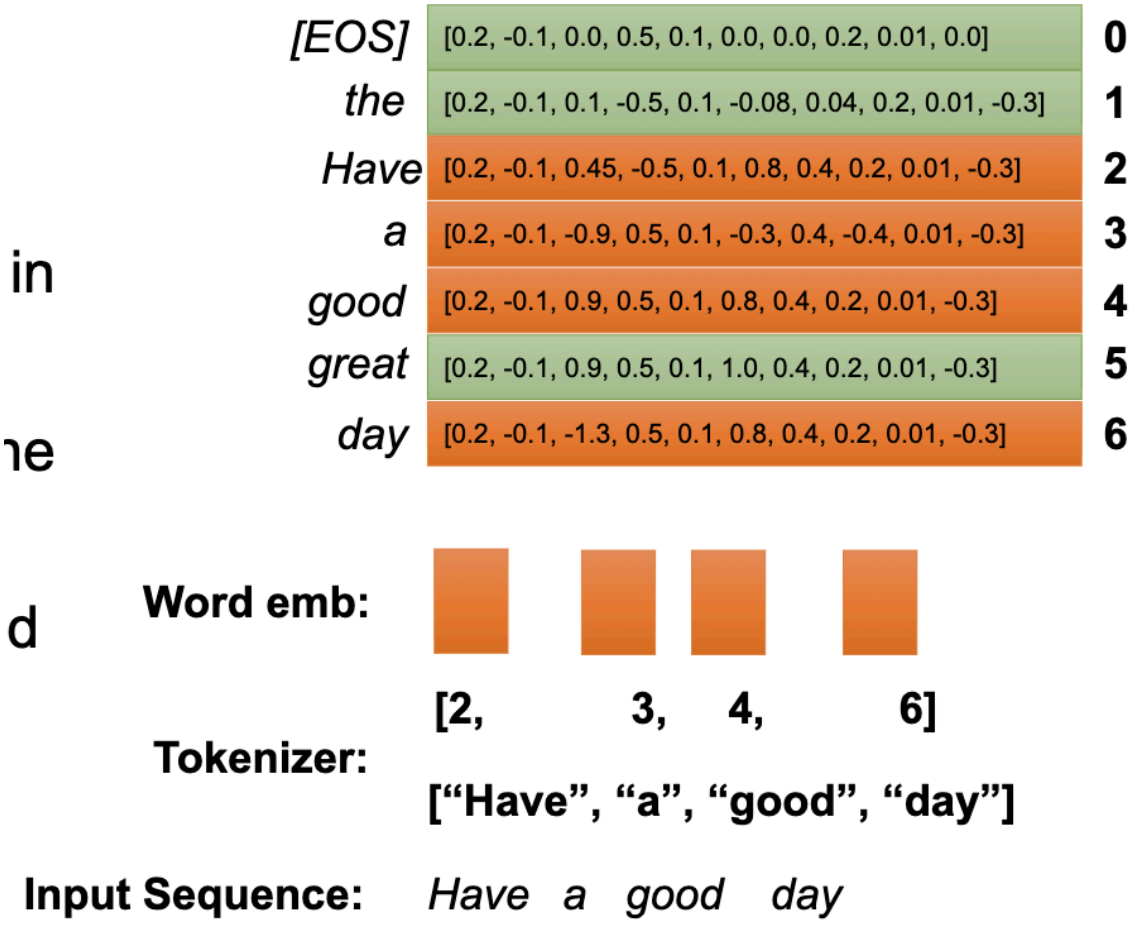

Figure 1: Embedding Table $\in \mathbb{R}^{V \times E}$

Table lookup is **much faster** as it avoids **BIG** matrix multiplications

Recurrent Neural Networks

family of neural networks for processing sequential data

- RNNs share parameters in a different way
  - Each member of output is a function of previous members of output
  - Each output produced using same update rule applied to previous outputs
  - This recurrent formulation results in sharing of parameters through a very deep computational graph

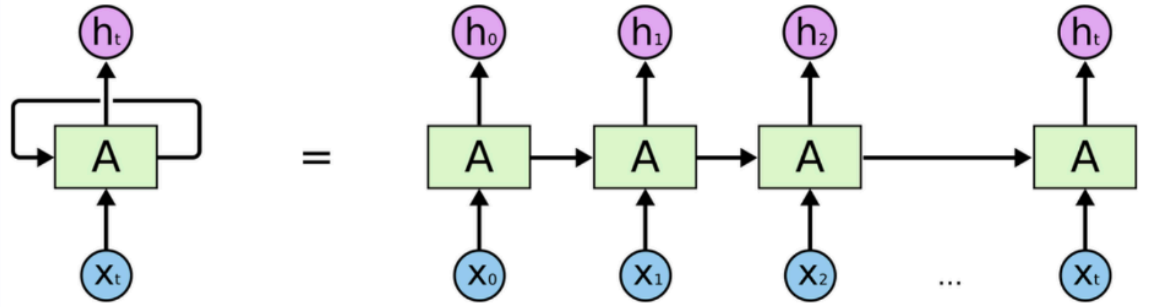

Figure 2: An unrolled RNN

RNNs share same weights across Time Step

To go from multi-layer networks to RNNs:

- Need to share parameters across different parts of a model
- Separate parameters for each value of input cannot generalize to sequence lengths not seen during training
- Share statistical strength across different sequence lengths and across different positions in time

Problem of Long-Term Dependencies

2 examples

- Easy to predict last word in “the clouds are in the **sky**”
- I grew up in France…[an entire story]… I speak fluent **French**.”

In principle RNNs should handle it, but fail in practice. LSTMs offer a solution

Backpropagation-though-time

Unfold the gomputational graph

A Computational Graph is a way to formalize the structure of a set of computations
(Backpropagation in this case)

- Unfolding this graph results in sharing of parameters across a deep network structure

Say that $s^{(t)} = f\left(s^{(t-1)}, \theta\right)$

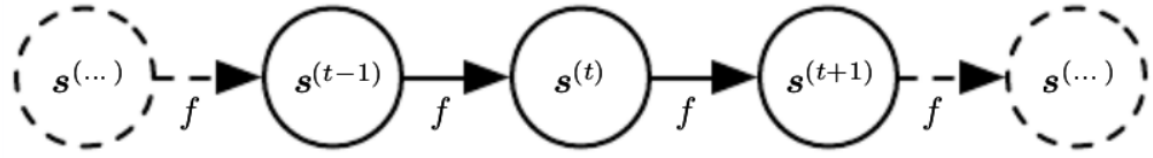

Figure 3: Classical dynamical system

Lets include the last state as well

$$s^{(t)} = f\left(s^{(t-1)}, x^{(t)}, \theta\right)$$

Now state now contains information about the whole past input sequence

Three design patterns of RNNs

Design 1

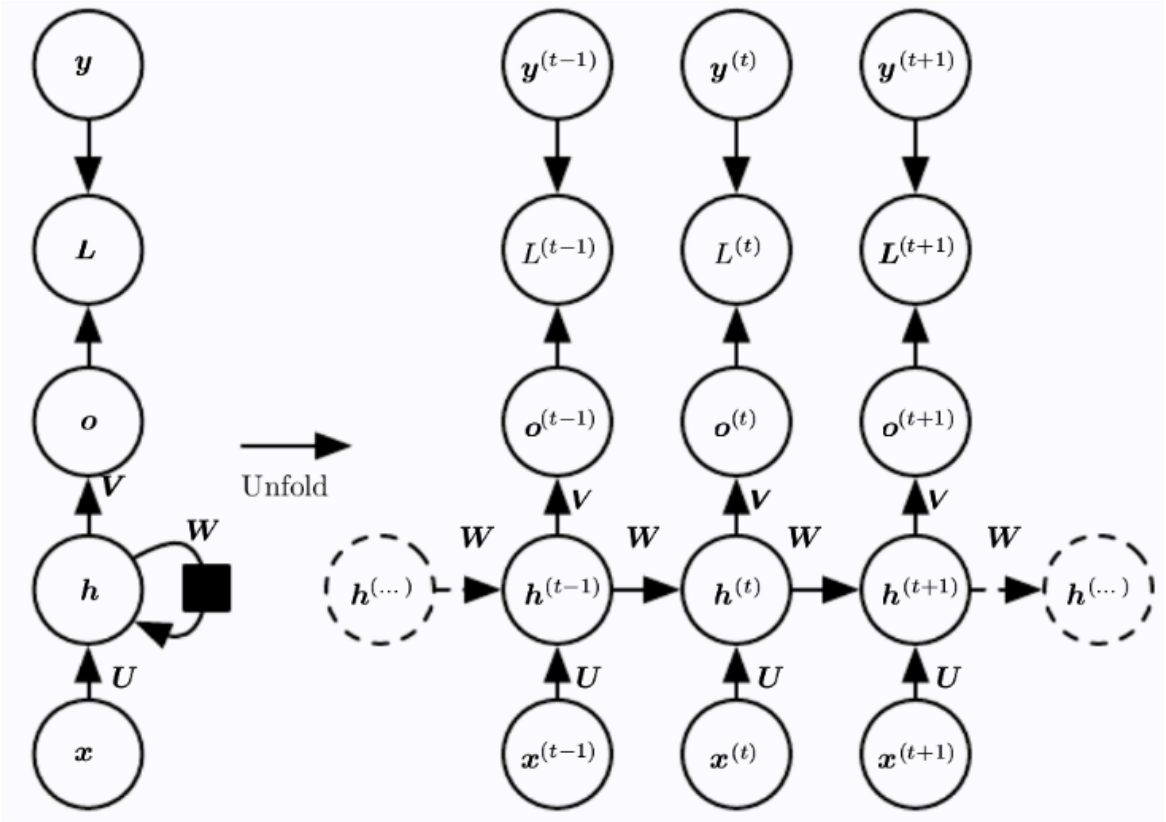

Figure 4: Design 1: Output: each time step; Recurrence: hidden units

Forward Calculation

We need to specify initial state $h^{(0)}$, then for each time point:

$$a^{(t)} = b0Wh^{(t-1)} + Ux^{(t)}$$

$$h^{(t)} = \tanh\left(a^{(t)}\right)$$

$$o^{(t)} = c + Vh^{(t)}$$

$$\hat{y}^{(t)} = \text{softmax}\left(o^{(t)}\right)$$

The Loss function can simply defined as the sum of the loss at each timepoint:

$$L = \sum_t L^{(t)} = -\sum_t \log p_{\text{model}(y^{(t)} \mid \{x^{(1)},\ldots,x^{(t)}\})}$$

Gradient Calculation (backward)

Computing this loss function $w.r.t\theta$ is very expensice

Requires forward propagation pass through the unrolled graph

Runtime is $\mathcal{O}(\tau)$ and cannot be reduced by parallelization

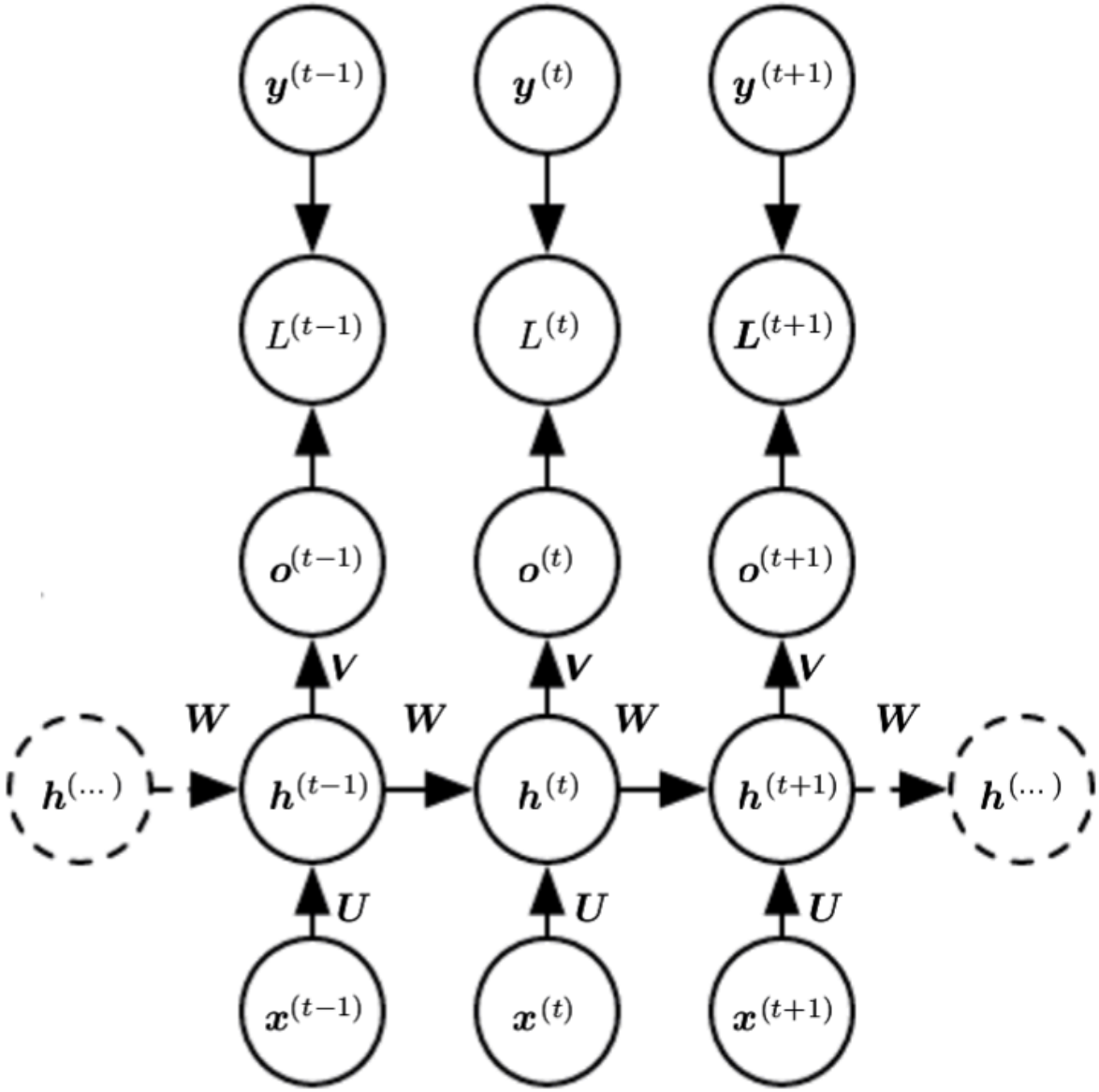

Figure 5: back-propagation through time or BPTT

Design 2

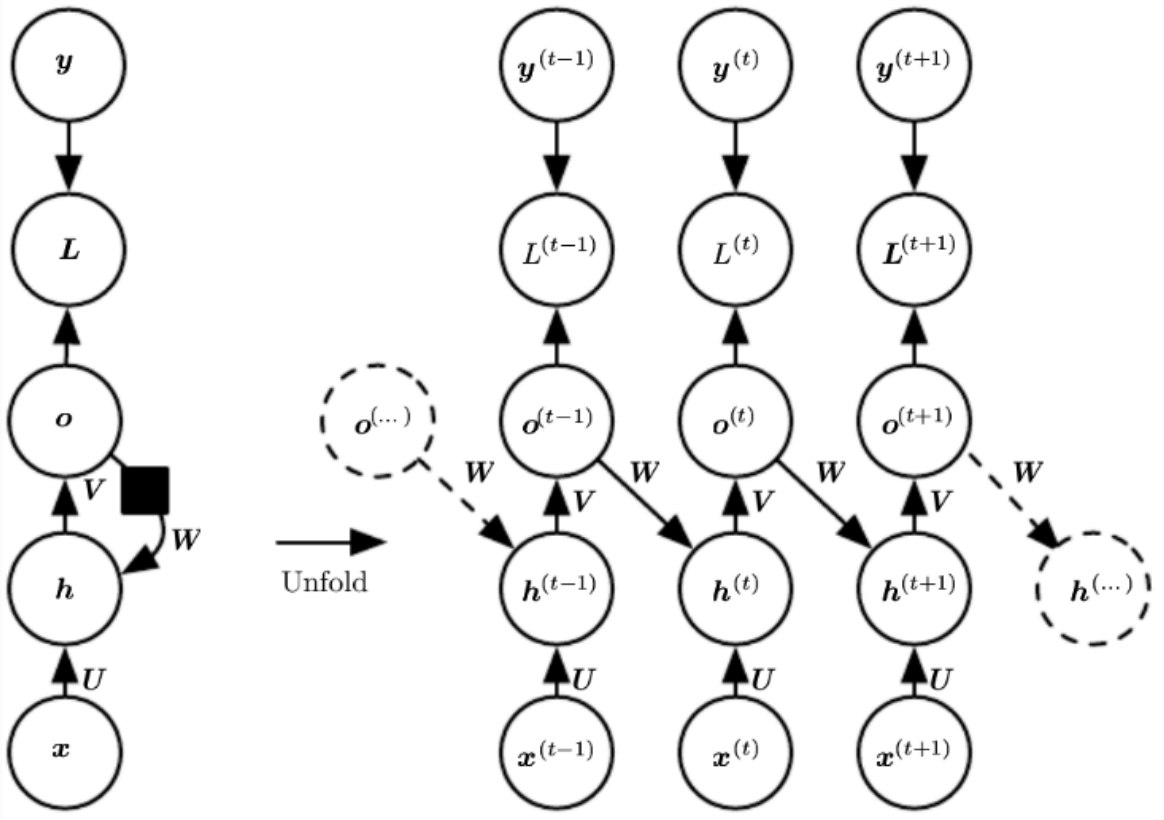

Figure 6: Design 2: Output: each time step; Recurrence: output units

Recurrence from Output to Hidden

Design 3

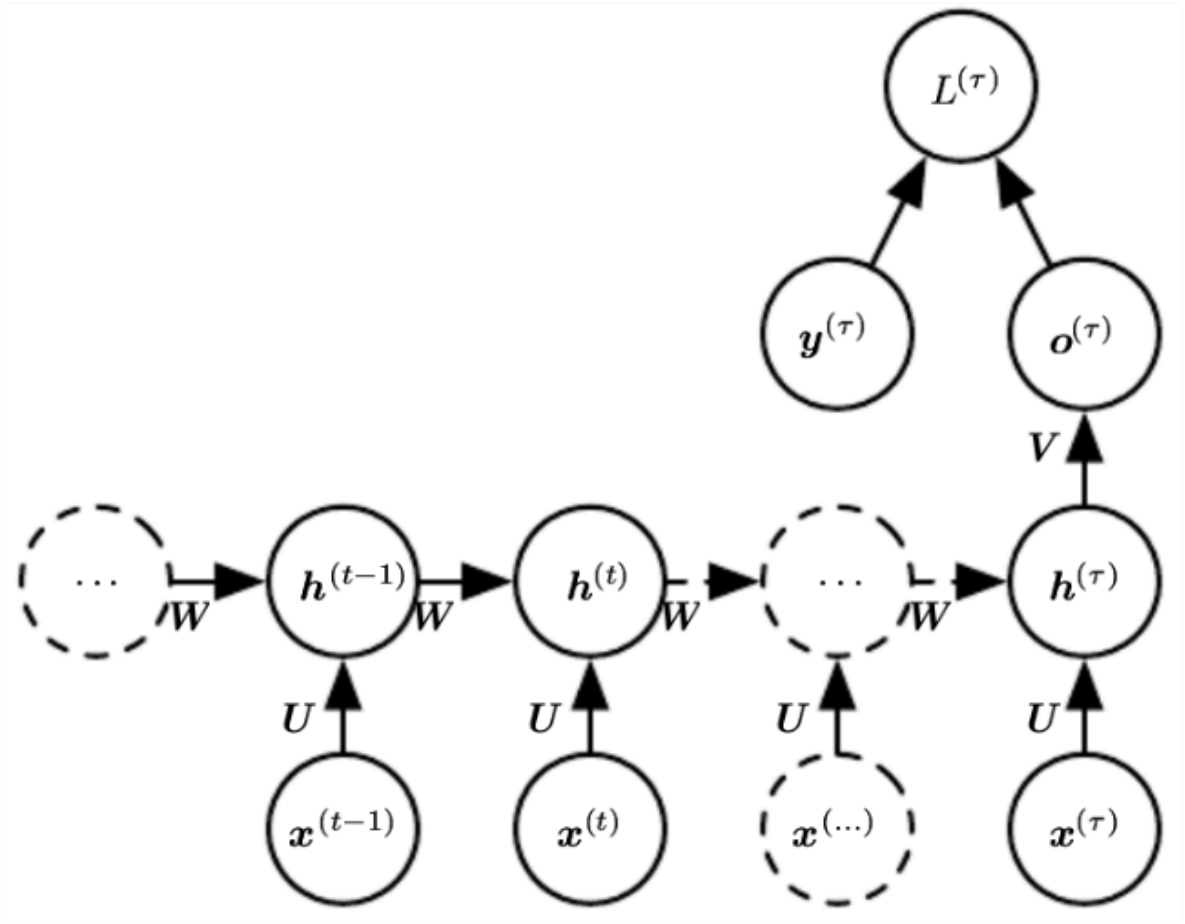

Figure 7: Design 3: Output: one at the end; Recurrence: hidden units

Teacher Forcing

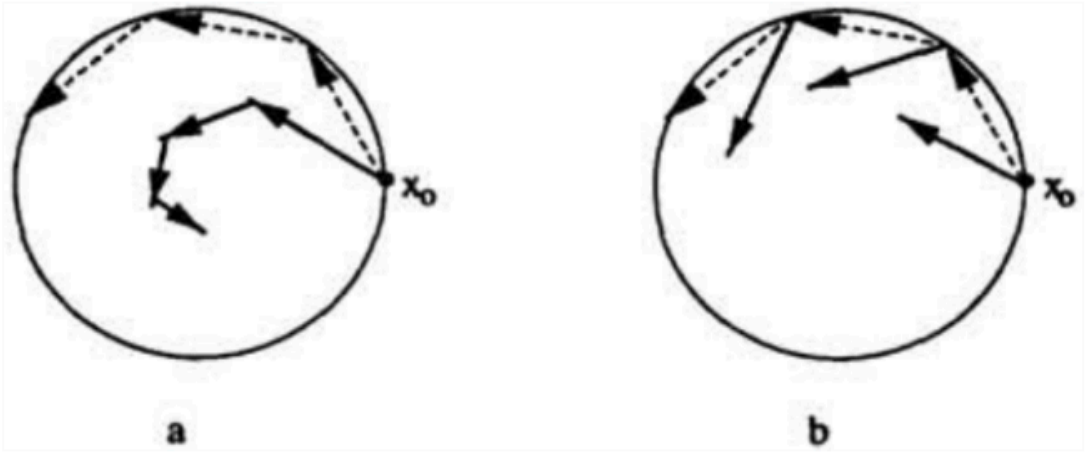

Figure 8: Visualizing Teacher Forcing

LSTM (long short-term memory)

LSTMs are explicitly designed to avoid the long-term dependency problem

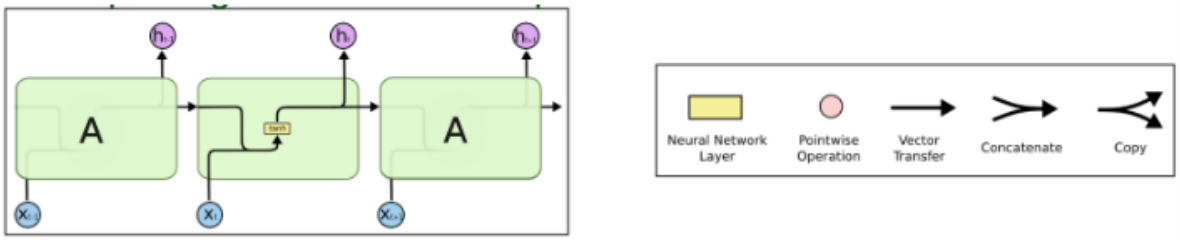

Figure 9: RNNs have the form of a repeating chain structure

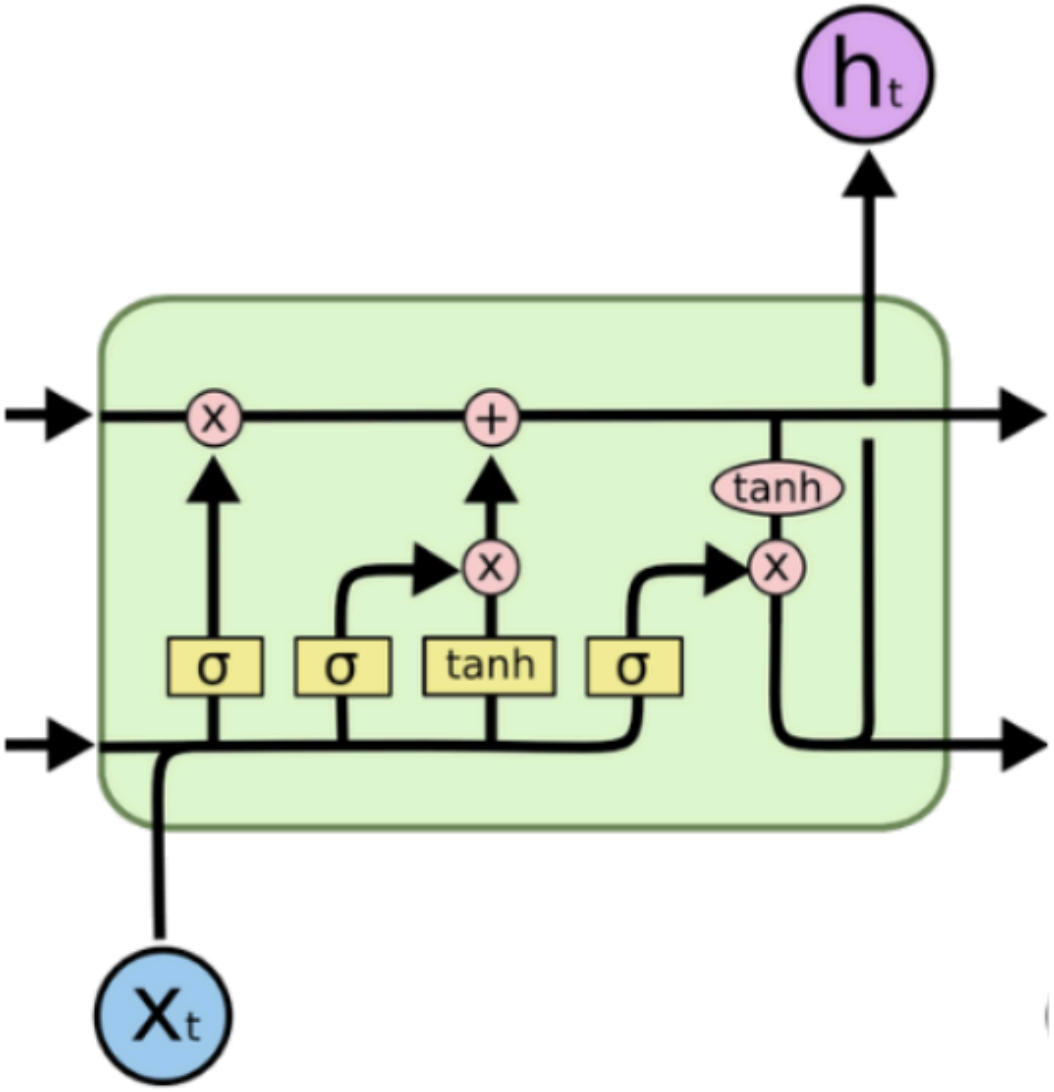

Figure 10: LSTMs also have a chain structure